

```

1: #include <stdio.h>
2: #include <string.h>
3: #include "i2p.h"
4:
5: #define TERMINATE 0
6: #define ADD_WORD 1
7: #define DISPLAY_WORDS 2
8: #define INCR_SORT 3
9: #define DECR_SORT 4
10: #define WORD_STATS 5
11: #define CHAR_STATS 6
12: #define SEARCH_WORD 7
13:
14: #define MAX_WORDS 30
15: #define MAX_WORD_LEN 31
16:
17: int menu(void);
18: int wordsLength;
19: int numOfWords;
20:
21: void addWord(void);
22: void displayWords(void);
23: void incSort(void);
24: void decSort(void);
25: void wordStats(void);
26: void charStats(void);
27: void searchWord(void);
28: void getStringV1(char message[], char str[]);
29:
30: void sortStringArrayInc(word, numOfWords, MAX_WORD_LEN);
31: void sortStringArrayDec(word, numOfWords, MAX_WORD_LEN);
32:
33: char word[MAX_WORDS][MAX_WORD_LEN];
34:
35: void main(){
36: int selectedOp;
37:
38: printf("WordsHandling\n\n");
39:     selectedOp=menu();
40:     while(selectedOp!= TERMINATE){
41:         switch(selectedOp){
42:             case ADD_WORD :
43:                 getNumOfWords("Enter number of words: ");
44:                 addWord();
45:                 break;
46:             case DISPLAY_WORDS :
47:                 displayWords();
48:                 break;
49:             case INCR_SORT :
50:                 incSort();

```

```

51:             break;
52:         case DECR_SORT :
53:             decSort();
54:             break;
55:         case WORD_STATS :
56:             wordStats();
57:             break;
58:         case CHAR_STATS :
59:             charStats();
60:             break;
61:         case SEARCH_WORD :
62:             searchWord();
63:             break;
64:         default :
65:             printf("Operation is not supported\n");
66:             break;
67:     }
68:     selectedOp=menu();
69: }
70: printf("WordsHandling terminated\n");
71: return;
72: }
73:
74: int menu(void){
75: int choice;
76:
77: printf("\n\n-----MENU-----\n");
78: printf("0 - TERMINATE\n");
79: printf("1 - ADD WORD\n");
80: printf("2 - DISPLAY WORDS\n");
81: printf("3 - INCREMENTAL SORT\n");
82: printf("4 - DECREMENTAL SORT\n");
83: printf("5 - WORD STATISTICS\n");
84: printf("6 - CHARACTER STATISTICS\n");
85: printf("7 - SEARCH WORD \n-----\n");
86: printf("Select operation:");
87: scanf("%d",&choice);
88: return (choice);
89: }
90:
91: void addWord(void){
92:     int i;
93:     for(i=0;i<numOfWords;i++){
94:         printf("Add word: \n");
95:         scanf("%s", word[i]);
96:     }
97: }
98:
99: void displayWords(void){
100:     printf("Display words: \n");

```

```

101:     int j;
102:     for(j=0;j<numOfWords;j++)
103:         printf("%s\n", word[j]);
104:
105: }
106:
107: void incSort(void){
108:     int o=0;
109:     sortStringArrayInc(word, numOfWords, MAX_WORD_LEN);
110:     for(o=0;o<numOfWords;o++){
111:         printf("%s\n",word[o]);
112:     }
113:
114: }
115:
116: void decSort(void){
117:     sortStringArrayDec(word, numOfWords, MAX_WORD_LEN);
118:     int u=0;
119:     for(u=0;u<numOfWords;u++){
120:         printf("%s\n", word[u]);
121:     }
122:
123: }
124:
125: void wordStats(void){
126:     int u, o, max, min, overallLength;
127:     float avg;
128:     overallLength=strlen(word[0]);
129:     max=strlen(word[0]);
130:     min=strlen(word[0]);
131:     for(u=1;u<numOfWords;u++){
132:         if(strlen(word[u])>max){
133:             max=strlen(word[u]);
134:         }
135:         if(strlen(word[u])<min){
136:             min=strlen(word[u]);
137:         }
138:     }
139:     for(o=1;o<numOfWords;o++){
140:         overallLength+=strlen(word[o]);
141:     }
142:     avg=(float)overallLength/numOfWords;
143:     printf("Max word length = %d \nMin word length = %d \nAverage word length = %f",max,min,avg);
144: }
145:
146: void charstats(void) {
147:     char schar;
148:     int totalCount = 0;
149:     int maxCount = 0;
150:     int minCount = 0;

```

```

151:     int wordCounts[30];
152:     char maxWordchar[maxwordlen];
153:     char minWordchar[maxwordlen];
154:     float averageCount = 0;
155:
156:
157:     printf("Enter character to search: ");
158:     scanf("%c", &schar);
159:
160:
161:
162:     minCount = strlen(word[0]);
163:
164:
165:
166:     for (i = 0; i < numOfWords; i++) {
167:         int currentCount = 0;
168:
169:
170:         int j;
171:         for (j = 0; j < strlen(word[i]); j++) {
172:             if (word[i][j] == schar) {
173:                 currentCount++;
174:             }
175:         }
176:
177:
178:         wordCounts[i] = currentCount;
179:         totalCount += currentCount;
180:
181:         if (currentCount > maxCount) {
182:             maxCount = currentCount;
183:             strcpy(maxWordchar, word[i]);
184:         }
185:         if (currentCount < minCount) {
186:             minCount = currentCount;
187:             strcpy(minWordchar, word[i]);
188:         }
189:     }
190:
191:     if (numOfWords > 0) {
192:         averageCount = (float)totalCount / numOfWords;
193:     }
194:
195:     printf("Total character results '%c':\n", schar);
196:     printf("Total character occurrences: %d\n", totalCount);
197:     printf("Average number of occurrences per word: %.2f\n", averageCount);
198:     if (numberOfWords > 0) {
199:         printf("Max number of occurrences in one word: %d (Word: '%s')\n", maxCount, maxWordchar);
200:         printf("Min number of occurrences in one word: %d (Word: '%s')\n", minCount, minWordchar);

```

```

201:     } else {
202:         printf("No words have been added to the list.\n");
203:     }
204: }
205:
206: void searchWord(void) {
207:     char sword[MAX_WORD_LEN];
208:     int foundcount = 0;
209:     int theseis[30];
210:
211:     getStringV1("Enter word to search: ", sword);
212:     int i;
213:     for (i = 0; i < numOfWorks; i++) {
214:         if (strcmp(word[i], sword) == 0) {
215:             theseis[foundcount] = i + 1;
216:             foundcount++;
217:         }
218:     }
219:
220:     if (foundcount > 0) {
221:         printf("Word '%s' found %d times on list.\n", sword, foundcount);
222:         printf("Found in positions below: ");
223:         for (i = 0; i < foundcount; i++){
224:             printf("%d ", theseis[i]);
225:         }
226:         printf("\n");
227:     } else {
228:         char choice;
229:         printf("Word '%s' not found.\n", sword);
230:         printf("Add word to list? (y/n)\n(y stands for yes and n stands for no,choose any key)");
231:         scanf(" %c", &choice);
232:
233:         if (choice == 'y'){
234:             if (numOfWorks < 30) {
235:                 strcpy(word[numOfWorks], sword);
236:                 numOfWorks++;
237:                 printf("Word '%s' added to list.\n", sword);
238:             } else {
239:                 printf("Word not added, list is full.\n");
240:             }
241:         } else {
242:             printf("Word not added.\n");
243:         }
244:     }
245: }
246:
247: int getNumOfWorks(char message[]) {
248:     printf(message);
249:     scanf("%d", &numOfWorks);
250:     return numOfWorks;

```

```
251: }  
252: int getInt(char message[]){  
253:     printf(message);  
254:     int num;  
255:     scanf("%d", &num);  
256:     return num;  
257: }
```